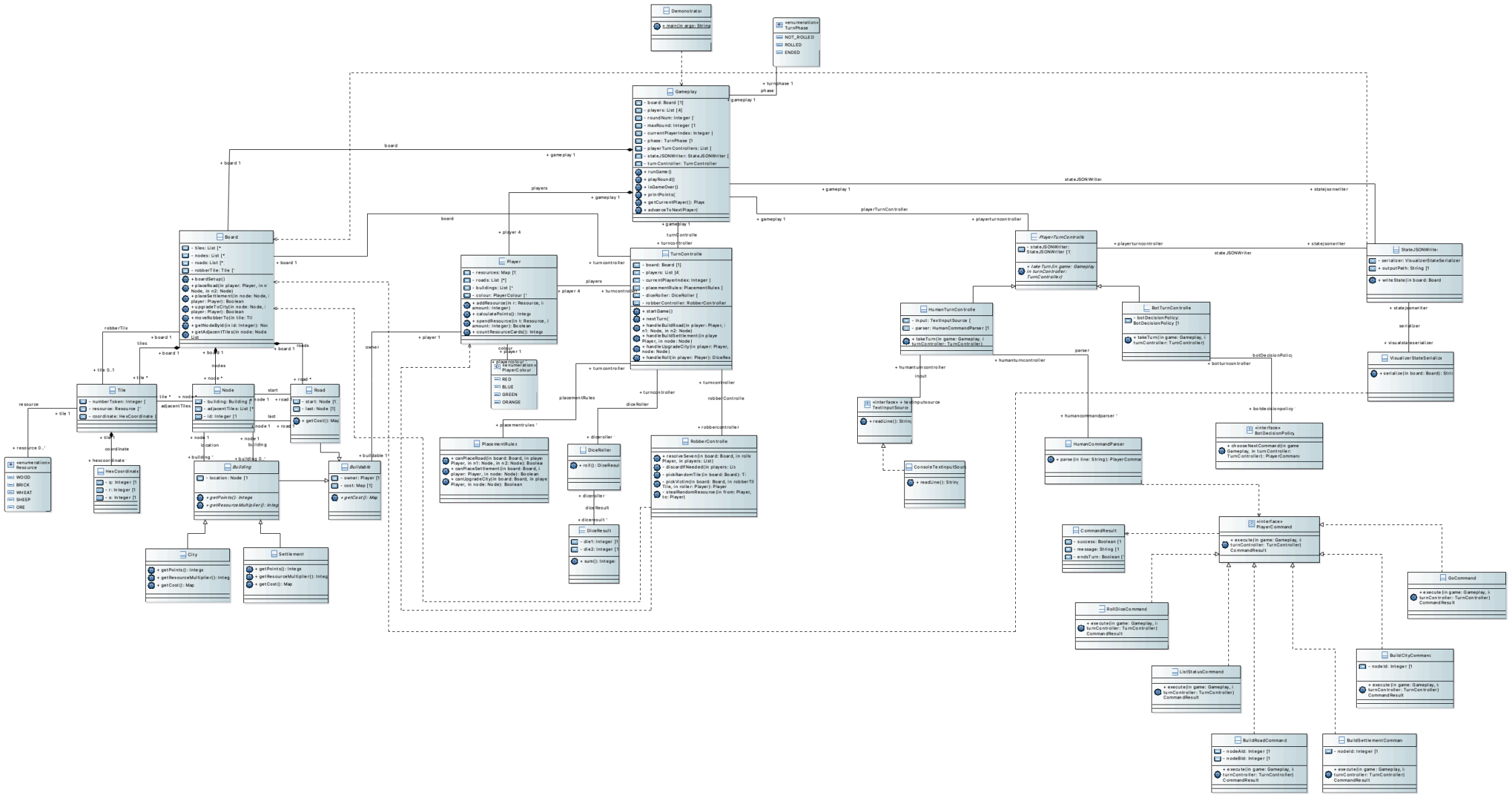
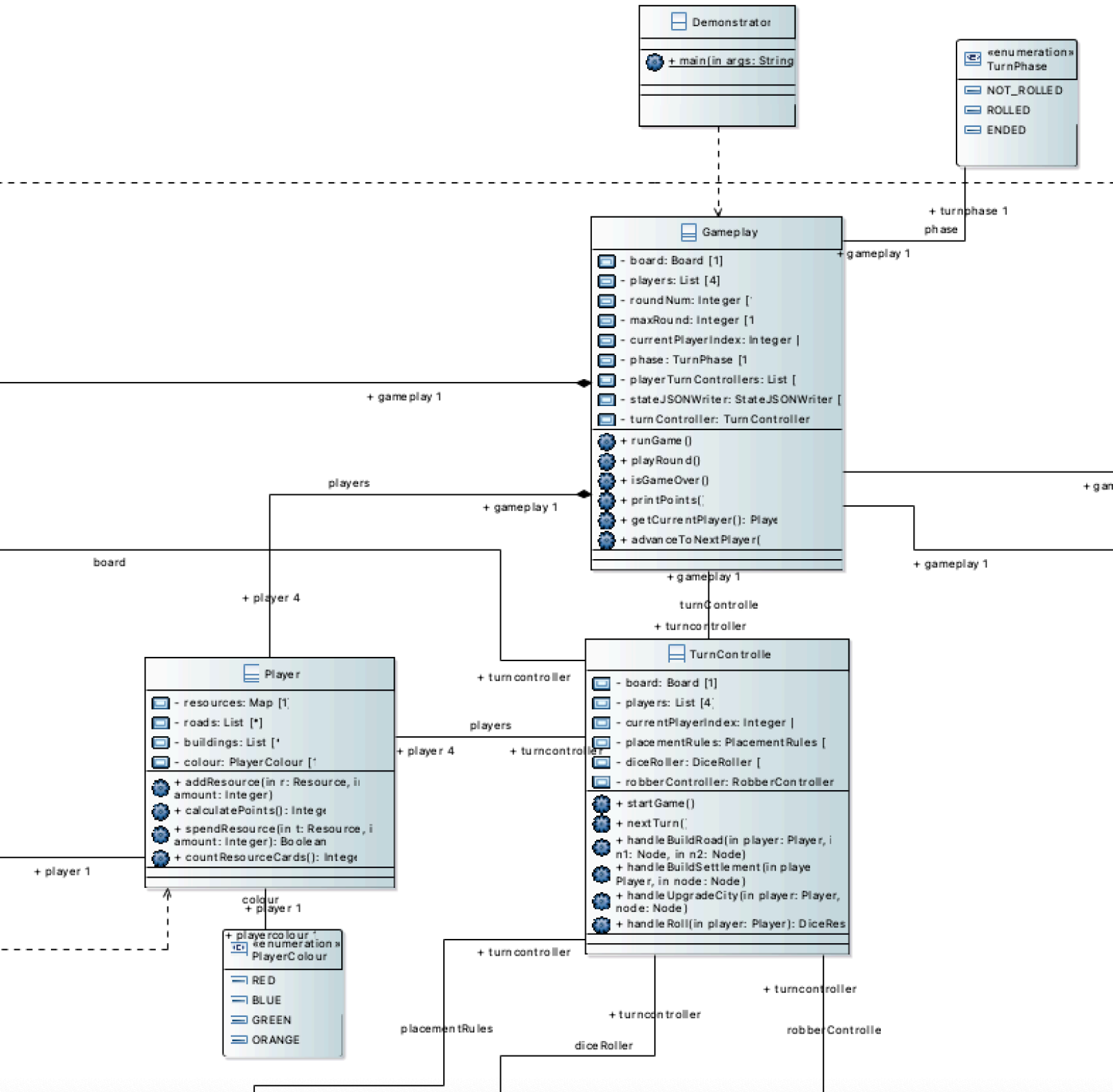
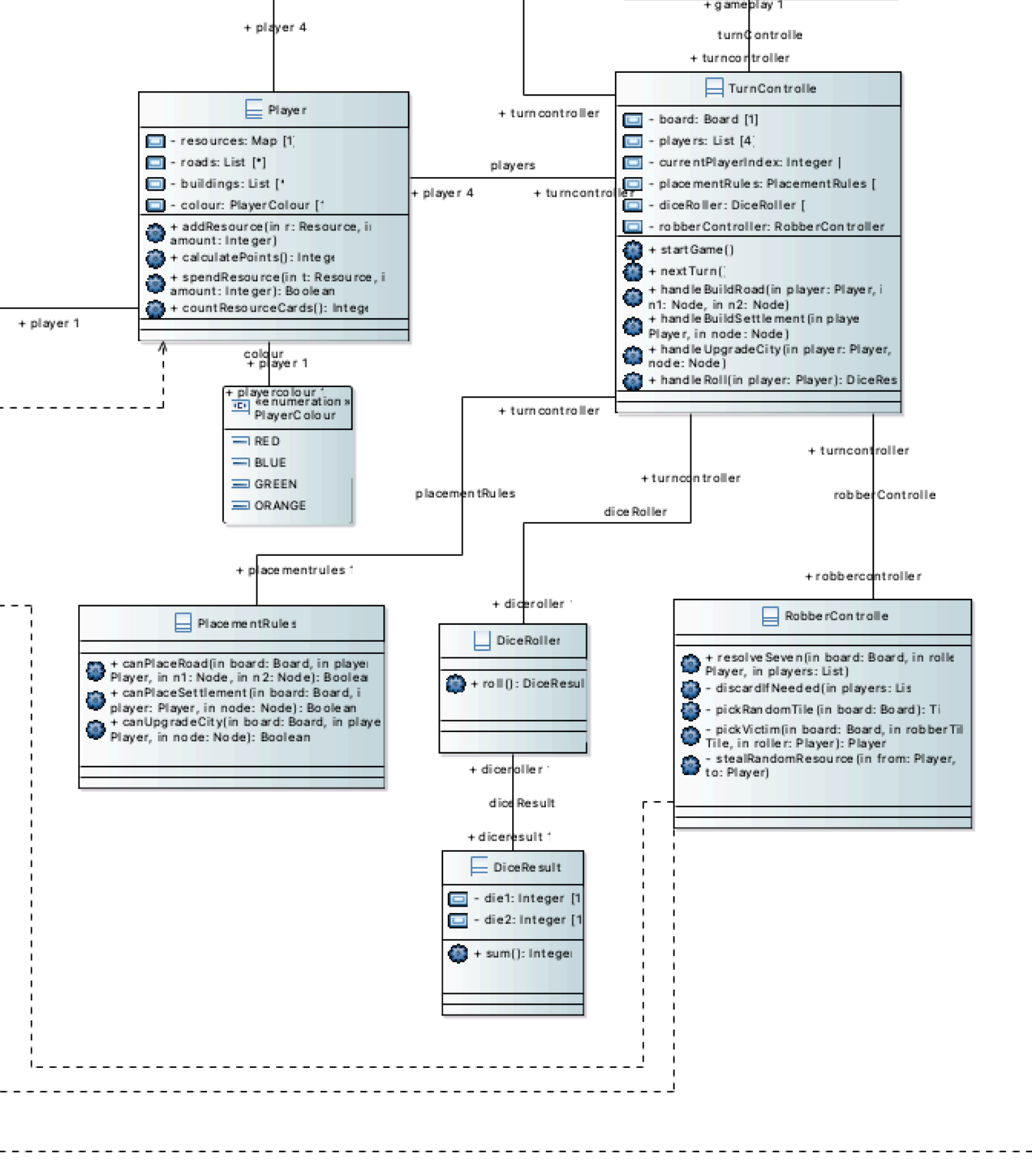
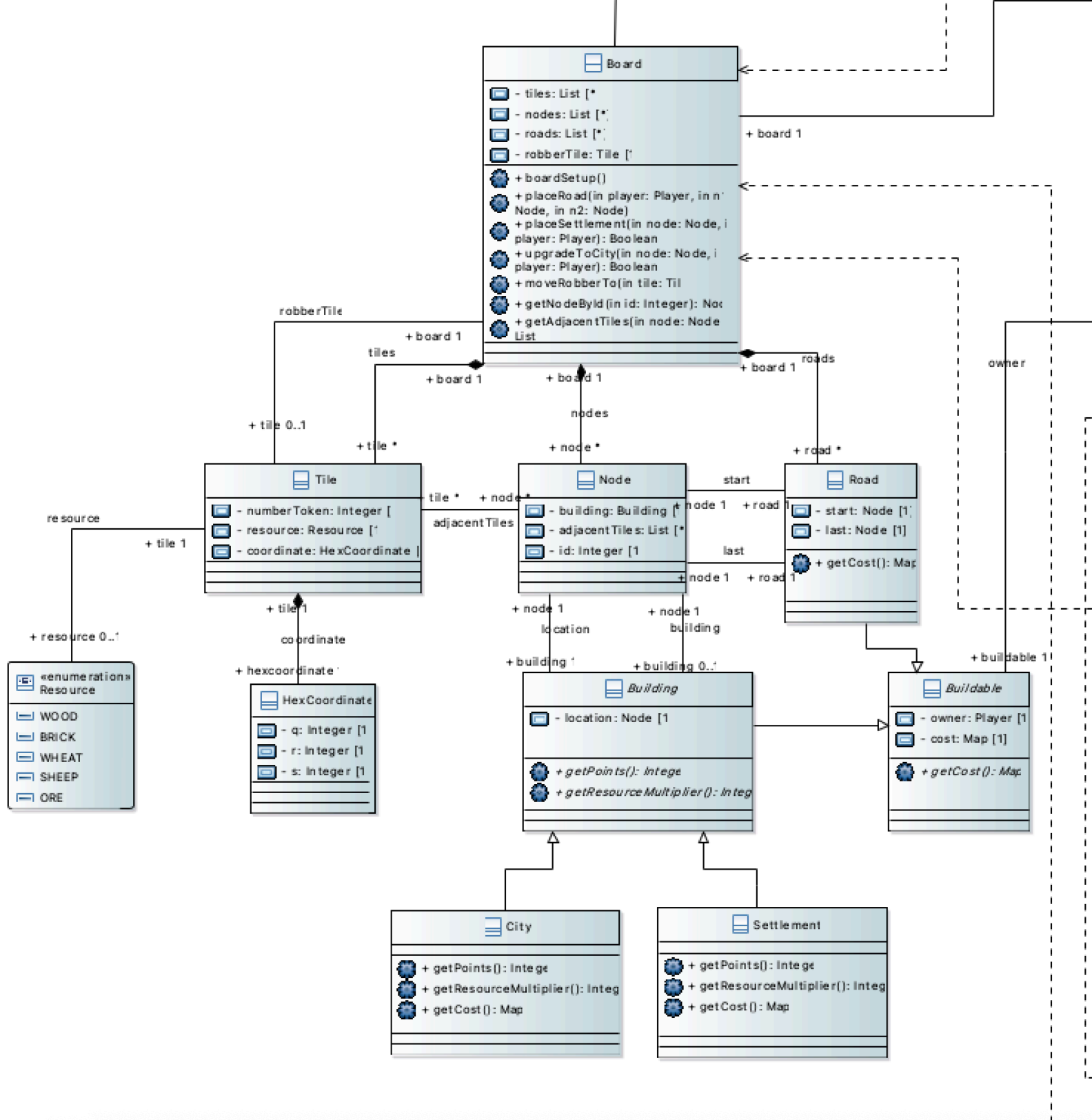
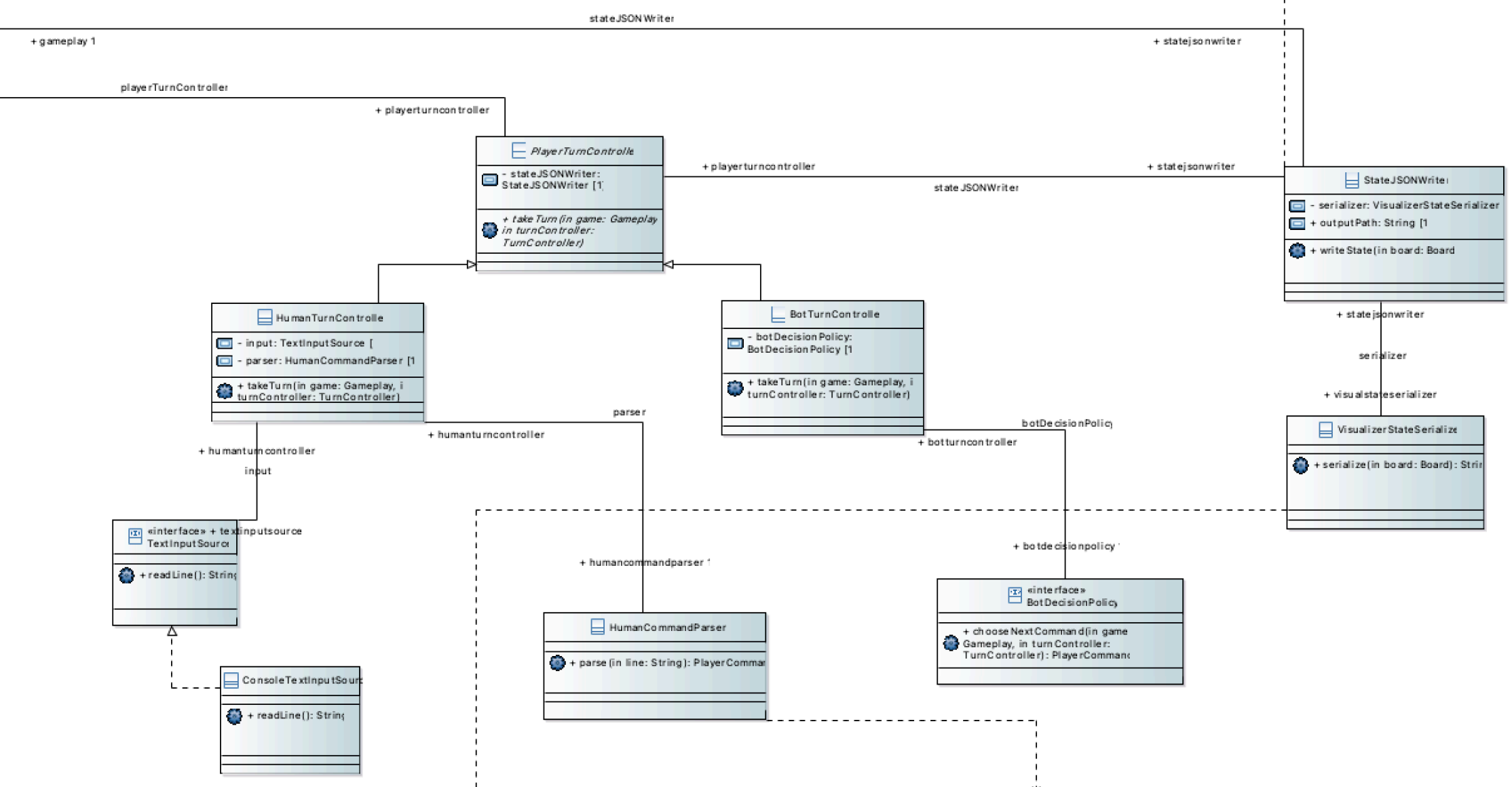
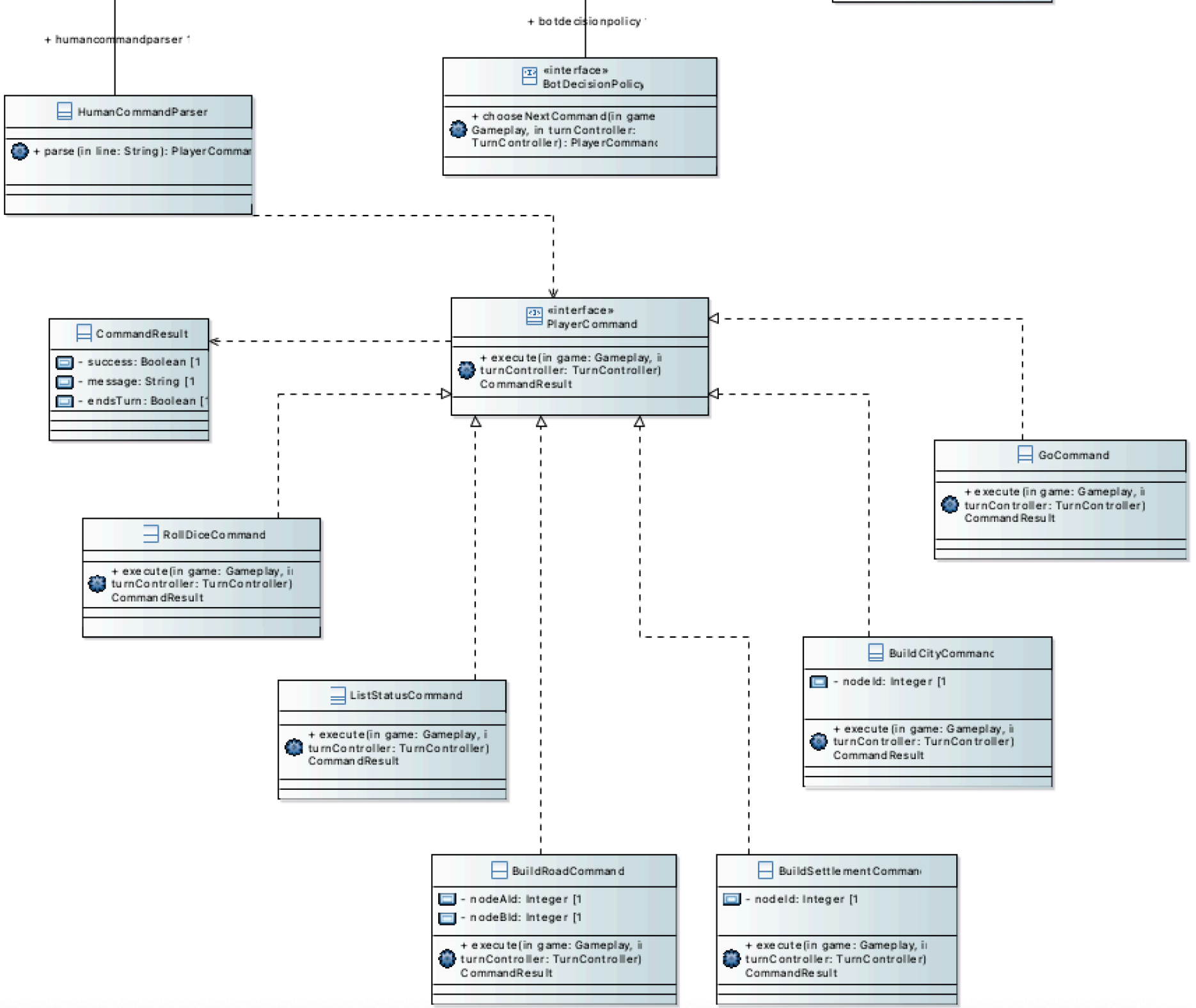

Demonstrator
+ main(in args: String
«enumeration» TurnPhase
NOT_ROLLED
ROLLED
ENDED
Gameplay
- board: Board [1]
- players: List [4]
- roundNum: Integer [
- maxRound: Integer [1
- currentPlayerIndex: Integer [
- phase : TurnPhase [1
- playerTurnControllers: List [
- stateJSONWriter: StateJSONWriter [
- turnController: TurnController
+ runGame()
+ playRound()
+ isGameOver()
+ printPoints(
+ getCurrentPlayer(): Playe
+ advanceToNextPlayer(
Board
- tiles: List [*
- nodes: List [*
- roads: List [*
- robberTile: Tile [
+ boardSetup()
+ placeRoad(in player: Player, in n
Node, in n2: Node)
+ placeSettlement(in node: Node, i
player: Player): Boolean
+ upgradeToCity(in node: Node, i
player: Player): Boolean
+ moveRobberTo(in tile: Til
+ getNodeById(in id: Integer): Noc
+ getAdjacentTiles(in node: Node
List
Player
- resources: Map [1
- roads: List [*]
- buildings: List [*
- colour: PlayerColour [
+ addResource(in r: Resource, i
amount: Integer)
+ calculatePoints(): Integ
+ spendResources(in t: Resource, i
amount: Integer): Boolean
+ countResourceCards(): Integ
TurnController
- board: Board [1]
- players: List [4
- currentPlayerIndex: Integer [
- placementRules: PlacementRules [
- diceRoller: DiceRoller [
- robberController: RobberController
+ startGame()
+ nextTurn()
+ handleBuildRoad(in player: Player, i
n1: Node, in n2: Node)
+ handleBuildSettlement(in playe
Player, in node: Node)
+ handleUpgradeCity(in player: Player,
node: Node)
+ handleRoll(in player: Player): DiceRes
PlayerTurnControlle
- stateJSONWriter:
StateJSONWriter [1
+ takeTurn(in game: Gameplay
in turnController:
TurnController)
HumanTurnControlle
- input: TextInputSource [
- parser: HumanCommandParser [1
+ takeTurn(in game: Gameplay, i
turnController: TurnController)
BotTurnControlle
- botDecisionPolicy:
BotDecisionPolicy [1
+ takeTurn(in game: Gameplay, i
turnController: TurnController)
StateJSONWrite
- serializer: VisualizerStateSerializer
+ outputPath: String [1
+ writeState(in board: Board
VisualizerStateSerializ
+ serialize(in board: Board): Strin
«enumeration» PlayerColour
RED
BLUE
GREEN
ORANGE
Tile
- numberToken: Integer [
- resource: Resource [
- coordinate: HexCoordinate
Node
- building: Building [
- adjacentTiles: List [*
- id: Integer [1
Road
- start: Node [1
- last: Node [1]
+ getCost(): Map
«enumeration» Resource
WOOD
BRICK
WHEAT
SHEEP
ORE
HexCoordinate
- q: Integer [1]
- r: Integer [1]
- s: Integer [1]
Building
- location : Node [1
+ getPoints(): Integ
+ getResourceMultiplier(): Integ
Buildable
- owner: Player [1]
- cost: Map [1]
+ getCost(): Map
City
+ getPoints(): Integ
+ getResourceMultiplier(): Integ
+ getCost(): Map
Settlement
+ getPoints(): Integ
+ getResourceMultiplier(): Integ
+ getCost(): Map
PlacementRules
+ canPlaceRoad(in board: Board, in playe
Player, in n1: Node, in n2: Node): Boolea
+ canPlaceSettlement(in board: Board, i
player: Player, in node: Node): Boolean
+ canUpgradeCity(in board: Board, in playe
Player, in node: Node): Boolean
DiceRoller
+ roll(): DiceResul
DiceResult
- die1: Integer [1
- die2: Integer [1
+ sum(): Integer
RobberController
+ resolveSeven(in board: Board, in rolle
Player, in players: List)
- discardIfNeeded(in players: Lis
- pickRandomTile(in board: Board): Ti
- pickVictim(in board: Board, in robberTil
Tile, in roller: Player): Player
- stealRandomResource(in from: Player,
to: Player)
«interface» TextInputSource
+ readLine(): Strin
ConsoleTextInputSour
+ readLine(): Strin
HumanCommandParser
+ parse(in line: String): PlayerComma
«interface» BotDecisionPolicy
+ chooseNextCommand(in game
Gameplay, in turnController:
TurnController): PlayerComman
«interface» PlayerCommand
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
CommandResult
- success: Boolean [1
- message: String [1
- endsTurn: Boolean [
GoCommand
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
RollDiceCommand
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
ListStatusCommand
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
BuildCityCommand
- nodeId: Integer [1
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
BuildRoadCommand
- nodeAId: Integer [1
- nodeBId: Integer [1
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
BuildSettlementComman
- nodeId: Integer [1
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
+ turnphase 1
phase
+ gameplay 1
board
+ board 1
+ gameplay 1
stateJSONWriter
+ statejsonwriter
+ gameplay 1
playerTurnController
+ gameplay 1
+ playerturncontroller
+ gameplay 1
turnControlle
+ turncontroller
+ turncontroller
+ player 4
+ turncontroller
+ playerturncontroller
+ statejsonwriter
state JSONWriter
+ statejsonwriter
serializer
+ visualizerserializer
+ board 1
+ board 1
+ board 1
+ board 1
+ board 1
+ board 1
+ board 1
+ board 1
+ board 1
+ board 1
+ board 1
robberTile
tiles
nodes
roads
+ tile 0..1
+ tile *
+ node *
+ road *
resource
+ tile 1
tile *
+ node *
adjacentTiles
start
+ road 1
last
+ node 1
+ road 1
+ resource 0..*
+ start
coordinate
+ hexcoordinate
+ building *
location
+ building 0..*
building
+ buildable 1
+ node 1
+ node 1
owner
+ player 1
colour
+ player 1
+ turncontroller
+ turncontroller
+ turncontroller
+ turncontroller
+ turncontroller
+ turncontroller
+ humanturncontroller
input
+ humanturncontroller
parser
+ humancommandparser *
+ botdecisionpolicy *
botDecisionPolicy
+ botturncontroller
+ placementrules *
placementRules
diceRoller
robberController
+ diceroller *
+ robbercontroller
+ diceroller *
diceResult
+ diceresult *


Demonstrator
+ main(in args: String
«enumeration»
TurnPhase
NOT_ROLLED
ROLLED
ENDED
+ turnphase 1
phase
Gameplay
+ gameplay 1
- board: Board [1]
- players: List [4]
- roundNum: Integer [
- maxRound: Integer [1
- currentPlayerIndex: Integer [
- phase: TurnPhase [1
- playerTurnControllers: List [
- stateJSONWriter: StateJSONWriter [
- turnController: TurnController
+ runGame()
+ playRound()
+ isGameOver()
+ printPoints(
+ getCurrentPlayer(): Playe
+ advanceToNextPlayer(
+ gameplay 1
players
+ gameplay 1
board
+ gameplay 1
+ gameplay 1
+ player 4
turnControlle
+ turncontroller
Player
- resources: Map [1
- roads: List [*]
- buildings: List [*
- colour: PlayerColour [*
+ addResource(in r: Resource, i
amount: Integer)
+ calculatePoints(): Intege
+ spendResource(in t: Resource, i
amount: Integer): Boolean
+ countResourceCards(): Intege
+ turncontroller
TurnControlle
- board: Board [1]
- players: List [4
- currentPlayerIndex: Integer [
- placementRules: PlacementRules [
- diceRoller: DiceRoller [
- robberController: RobberController
+ startGame()
+ nextTurn(
+ handleBuildRoad(in player: Player, i
n1: Node, in n2: Node)
+ handleBuildSettlement(in playe
Player, in node: Node)
+ handleUpgradeCity(in player: Player,
node: Node)
+ handleRoll(in player: Player): DiceRes
players
+ player 4
+ turncontroller
+ player 1
colour
+ player 1
+ playercolour *
«enumeration»
PlayerColour
RED
BLUE
GREEN
ORANGE
+ turncontroller
+ turncontroller
+ turncontroller
placementRules
dice Roller
robberControlle


+ player 4
+ gameplay 1
turnControlle
+ turncontroller
Player
- resources: Map [1
- roads: List [*]
- buildings: List [*
- colour: PlayerColour [1
+ addResource(in r: Resource, i
amount: Integer)
+ calculatePoints(): Intege
+ spendResource(in t: Resource, i
amount: Integer): Boolean
+ countResourceCards(): Integ
+ turncontroller
TurnControlle
- board: Board [1]
- players: List [4
- currentPlayerIndex: Integer [
- placementRules: PlacementRules [
- diceRoller: DiceRoller [
- robberController: RobberController
+ startGame()
+ nextTurn(
+ handleBuildRoad(in player: Player, i
n1: Node, in n2: Node)
+ handleBuildSettlement(in playe
Player, in node: Node)
+ handleUpgradeCity(in player: Player,
node: Node)
+ handleRoll(in player: Player): DiceRes
players
+ player 4
+ turncontroller
+ player 1
colour
+ player 1
+ playercolour 1
«enumeration»
PlayerColour
RED
BLUE
GREEN
ORANGE
+ turncontroller
+ turncontroller
+ turncontroller
placementRules
diceRoller
robberControlle
+ placementrules 1
+ robbercontroller
+ diceroller 1
PlacementRules
+ canPlaceRoad(in board: Board, in playe
Player, in n1: Node, in n2: Node): Boolea
+ canPlaceSettlement(in board: Board, i
player: Player, in node: Node): Boolean
+ canUpgradeCity(in board: Board, in playe
Player, in node: Node): Boolean
DiceRoller
+ roll(): DiceResul
RobberControlle
+ resolveSeven(in board: Board, in rolle
Player, in players: List)
- discardIfNeeded(in players: Lis
- pickRandomTile(in board: Board): Ti
- pickVictim(in board: Board, in robberTil
Tile, in roller: Player): Player
- stealRandomResource(in from: Player,
to: Player)
+ diceroller 1
diceResult
+ diceresult 1
DiceResult
- die1: Integer [1
- die2: Integer [1
+ sum(): Integer


Board
- tiles: List [*
- nodes: List [*
- roads: List [*
- robberTile: Tile [
+ boardSetup()
+ placeRoad(in player: Player, in n
Node, in n2: Node)
+ placeSettlement(in node: Node, i
player: Player): Boolean
+ upgradeToCity(in node: Node, i
player: Player): Boolean
+ moveRobberTo(in tile: Til
+ getNodeById(in id: Integer): No
+ getAdjacentTiles(in node: Node
List
+ board 1
robberTile
+ board 1
tiles
+ board 1
+ board 1
+ board 1
roads
owner
+ tile 0..1
+ tile *
nodes
+ node *
+ road *
Tile
- numberToken: Integer [
- resource: Resource [
- coordinate: HexCoordinate
tile *
+ node *
adjacentTiles
Node
- building: Building [
- adjacentTiles: List [*
- id: Integer [1
start
+ node 1
+ road 1
last
+ node 1
+ road 1
Road
- start: Node [1
- last: Node [1]
+ getCost(): Map
resource
+ tile 1
+ tile 1
coordinate
+ hexcoordinate
+ resource 0..1
«enumeration»
Resource
WOOD
BRICK
WHEAT
SHEEP
ORE
HexCoordinate
- q: Integer [1
- r: Integer [1
- s: Integer [1
+ node 1
location
+ node 1
building
+ building
+ building 0..
Building
- location: Node [1
+ getPoints(): Intege
+ getResourceMultiplier(): Integ
+ buildable 1
Buildable
- owner: Player [1
- cost: Map [1]
+ getCost(): Map
City
+ getPoints(): Intege
+ getResourceMultiplier(): Integ
+ getCost(): Map
Settlement
+ getPoints(): Intege
+ getResourceMultiplier(): Integ
+ getCost(): Map


stateJSONWriter
+ gameplay 1
+ statejsonwriter
playerTurnController
+ playerturncontroller
PlayerTurnControlle
- stateJSONWriter: StateJSONWriter [1
+ takeTurn(in game: Gameplay in turnController: TurnController)
+ playerturncontroller
+ statejsonwriter
stateJSONWriter
StateJSONWrite
- serializer: VisualizerStateSerializer
+ outputPath: String [1
+ writeState(in board: Board
HumanTurnControlle
- input: TextInputSource [
- parser: HumanCommandParser [1
+ takeTurn(in game: Gameplay, i turnController: TurnController)
BotTurnControlle
- botDecisionPolicy: BotDecisionPolicy [1
+ takeTurn(in game: Gameplay, i turnController: TurnController)
+ statejsonwriter
serializer
+ visualstateserializer
VisualizerStateSerializ
+ serialize(in board: Board): Strin
parser
+ humanturncontroller
botDecisionPolic
+ botturncontroller
+ humanturncontroller
input
«interface» + textinputsource
TextInputSourc
+ readLine(): String
+ humancommandparser
+ botdecisionpolicy
«interface»
BotDecisionPolicy
+ chooseNextCommand(in game Gameplay, in turnController: TurnController): PlayerCommand
HumanCommandParser
+ parse(in line: String): PlayerComma
ConsoleTextInputSour
+ readLine(): String


+ humancommandparser
+ botdecisionpolicy
HumanCommandParser
+ parse (in line: String): PlayerCommar
«interface»
BotDecisionPolicy
+ chooseNextCommand(in game
Gameplay, in turnController:
TurnController): PlayerCommanc
«interface»
PlayerCommand
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
CommandResult
- success: Boolean [1
- message: String [1
- endsTurn: Boolean [1
GoCommand
+ execute (in game: Gameplay, i
turnController: TurnController)
CommandResult
RollDiceCommand
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
ListStatusCommand
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
BuildCityCommanc
- nodeId: Integer [1
+ execute (in game: Gameplay, i
turnController: TurnController)
CommandResult
BuildRoadCommand
- nodeAId: Integer [1
- nodeBId: Integer [1
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
BuildSettlementComan
- nodeId: Integer [1
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult